

Distilling Tree-of-Thought Reasoning into Small Language Models via Prompt-Guided Imitation

by

Student Name: Tasnuba Mehjabin

Student ID: 21301492

Student Name: Shah Faiyaz Zaman

Student ID: 21301097

Student Name: MD Mahmudun Nur Miraz

Student ID: 22201669

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
June 2026.

© 2025. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

(Signature)
Tasnuha Mehjabin
21301492

(Signature)
Shah Faiyaz Zaman
21301097

(Signature)
MD Mahmudun Nur Miraz
22201669

Approval

The thesis/project titled “Distilling Chain-of-Thought Reasoning into Small Language Models via Prompt-Guided Imitation ” submitted by

1. Tasnuba Mehjabin (21301492)
2. Shah Faiyaz Zaman (21301097)
3. MD Mahmudun Nur Miraz (22201669)

of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Spring 2026 Year.

Examining Committee:

Supervisor:
(Member)

Moin Mostakim

Senior Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam, PhD

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Ethics Statement (Optional)

This is optional, if you don't have an ethics statement then omit this page

Abstract

Large Language Models (LLMs) show improved reasoning performance when prompted to exhibit explicit reasoning structures, such as Chain-of-Thought (CoT). However, CoT is still inherently linear and autoregressive, which prevents systematic exploration, backtracking, and correcting immediate reasoning errors. These limitations were addressed by Tree-of-Thoughts(ToT) by using structured search over multiple reasoning paths, but came with the high cost of computational overhead. This overhead made it impractical for deployment on resource-constrained systems, particularly in Small Language Models (SLMs), which struggle to maintain long, consistent chains of reasoning and perform poorly with standard CoT prompting. In this work, we propose a teacher-student framework that distils structured Tree-of-Thought reasoning into Lightweight SLMs through One-Step Tree-of-Thought abstraction. We start by designing a multi-step ToT controller using a capable teacher LLM (gpt4o-mini), combining executable code verification, heuristic-based pruning, and learned value estimation to generate high-quality reasoning trees for multi-step reasoning tasks. From the successful solution paths, we extract and linearise the traces that show the planning decisions, intermediate evaluations, and corrective behaviour in a single step suitable for standard language-model finetuning. Using the distilled traces, we train SLMs (360M-2B parameters) using causal language modelling to mimic the teacher’s decision-making process without requiring explicit tree search during inference time. The resulting finetuned models are evaluated on successes, reasoning accuracy, generalisation when facing unseen problems, and computational efficiency, compared against Chain-of-Thought and Tree-of-Thought baselines. Our results demonstrate that structured ToT behaviour can be effectively transferred to smaller models through one-step distillation, resulting in improved reasoning systems by bridging the gap between powerful, but costly, LLM-based search and efficient, deployable SLMs.

Keywords: LLM, SLM, Tree-of-Thought (ToT), Chain-of-Thought (CoT)

Dedication (Optional)

A dedication is the expression of friendly connection or thanks by the author towards another person. It can occupy one or multiple lines depending on its importance. You can remove this page if you want.

Acknowledgement

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iii
Abstract	iv
Dedication	v
Acknowledgment	vi
Table of Contents	vii
List of Figures	ix
List of Tables	x
Nomenclature	x
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study or Motivation	1
1.3 Problem Statement	2
1.4 Objective	2
1.5 Methodology in Brief	3
1.6 Scopes and Challenges	3
2 Literature Review	5
2.1 Preliminaries	5
2.2 Review of Existing Research	6
2.2.1 From Linear Reasoning to Tree-Based Reasoning	6
2.2.2 Knowledge Distillation, Small Models, and Agentic Behavior .	7
2.2.3 Code-Grounded Verification and Search Reliability	7
2.2.4 Memory-Augmented Search and Tree Trace Generation	8
2.3 Summary of Key Findings	8
3 Requirements, Impacts and Constraints	10
3.1 Final Specifications and Requirements	10
3.2 Societal Impact	11

3.3	Environmental Impact	12
3.4	Ethical Issues	12
3.5	Standards	12
3.6	Project Management Plan	13
3.7	Risk Management	13
3.8	Economic Analysis	14
4	Proposed Methodology	15
4.1	Design Process or Methodology Overview	15
4.2	Preliminary Design or Design (Model) Specification	17
4.2.1	Stage 1: The Teacher (A Multi-Step Tree of Thought Solver) .	17
4.2.2	Stage 2: One-Step ToT Conversion Design	21
4.2.3	Stage 3: Student Training	22
5	Result Analysis	24
5.1	Performance Evaluation	24
5.2	Analysis of Design Solutions	24
5.3	Statistical Analysis	27
5.4	Comparisons and Relationships	28
5.5	Discussions	30
6	Conclusion	32
6.1	Summary of Findings	32
6.2	Contributions to the Field	32
6.3	Recommendations for Future Work	33
	Bibliography	35

List of Figures

4.1	CoT vs ToT.	16
4.2	Simplified pipeline.	17
4.3	One round of the teacher search.	18
4.4	The three alternative teacher designs.	19
4.5	A real raw search tree from our teacher for the puzzle “1 2 4 7”. . . .	20
4.6	Thought Decomposition.	20
4.7	One-step data augmentation pipeline.	21
5.1	Distilled student performance on the test set for Qwen, SmolLM (their paper), and SmolLM (our work).	25
5.2	Depth expansion and pruning across 100 test trees (puzzles 901–1000). 26	
5.3	Evaluation loss during training for SmolLM-360M and Qwen2.5-0.5B-Instruct.	26
5.4	Training loss during training for SmolLM-360M and Qwen2.5-0.5B-Instruct.	26
5.5	Training dataset size comparison between our work and the Babysitting paper.	28
5.6	All four results against the CoT baseline (7%, dashed). Every result clears it, but our teacher (89%) and student (24%) clear it far more decisively than the babysitting teacher (19%) and student (9%). . . .	30
5.7	The same results against the competing teacher’s ceiling ($\approx 19\%$, dashed). Our distilled student rises above the line and it solves more than the model that generated the rival dataset.	30

List of Tables

5.1	Success rate of distilled students on the same test dataset (puzzles 901–1000). Our best model Qwen performs at nearly double the success rate of the comparison result from previous work.	24
5.2	Teacher efficiency metrics averaged over 100 test puzzles.	27
5.3	Distribution of backtrack complexity across the 1,487 training examples.	27
5.4	Comparison paper results [11] (above dashed line) vs. our results (below) on test puzzles 901–1000.	29

Chapter 1

Introduction

1.1 Background

Artificial intelligence always struggled to reason like human, but things changed with a technique called Chain-of-Thought (CoT). This method helps large language models (LLMs) to solve problems by breaking them into a series of steps. Although this technique works like wonder for large models, the thing is the same "thinking out loud" in steps process rarely happens in smaller models specifically with fewer than 7 billion parameters. These small models simply lack the brainpower to develop these complex reasoning steps on their own, which is a major issue if we want to run smart AI on everyday gadgets like phones or cheap laptops.

To fix this issue, researchers have started using "distillation," the basic idea behind it is like a giant model tutoring a small one to pass down its knowledge. However, even with this help, most models still think in a linear direction. That's why when they make a single mistake at the beginning of a problem, they have no way to go back and fix it; they just keep going until they reach a dead end. This is where the **Tree of Thoughts (ToT)** framework comes in. Instead of a straight line, it allows a model to explore different branches of an idea, evaluate its own moves, and backtrack if it hits a dead end. Teaching a small model to handle this kind of non-linear, planned thinking is still a very new frontier in AI research.

1.2 Rationale of the Study or Motivation

Currently, Multi-Step ToT has the highest success rate in solving multipath problems. While it can achieve 82% success rate on Game of 24, a multipath problem, it requires dozens of GPT4o calls per puzzle. For a bounded problem, the cost-effectiveness is negligible, and even if it increases, it's predictable. But for unbounded multipath problems, token and API costs are unpredictable. Thus, at a large scale for both bounded and unbounded problems, multi-step ToT is not sustainable environmentally and financially. As such, researchers and institutions in low-resource environments cannot afford this. Small Language Models (SLMs) are Language Models with fewer than 1B parameters, and their usage has been rising due to their low cost and ease of deployment on edge hardware. But standard prompting techniques like CoT fail on multipath problems since they lack the arithmetic priors, and their context window capacity cannot sustain the heavy multi-step reasoning. Thus, a bridge is required to join and replicate the high success rate and

backtracking capability of multi-step ToT, the one-prompt capability of One-step ToT, and the knowledge distillation process. The key insight is that SLM is not required to run the whole tree search, unlike the teacher. The SLM only need to run in a linear path, and backtrack to the last stable step only when encountering a dead end. By collapsing full trees into a structured linear prompt and traces consisting of both clean and encountering dead ends, then correcting course, we can transfer the reasoning outcome without the large computation cost that comes from the multiple dead branches from multi-step ToT. The pipeline needs to be task-agnostic and can be adapted to any task that requires multipath reasoning.

1.3 Problem Statement

Chain-of-thought prompting improves the reasoning capability of LLMs, but because it follows a strict linear path, a mistake at one step cascades into failure, with no mechanism to backtrack. Tree-of-Thoughts addressed this, enabling multi-path exploration and achieving up to 82% success on Game of 24. But Tree-of-Thoughts costs dozens of LLM API calls per problem, which makes it computationally and financially impractical at a large scale. SLMs offer a more deployable and cost-effective alternative, but existing knowledge distillation approaches transfer only CoT traces to the student. This leads to the original problem of CoT, where the student never has the capacity to recover from a failed reasoning path. This leads to an unresolved gap: powerful tree-structured reasoning can only benefit large and expensive models, and SLMs are stuck with linear reasoning, which fails on problems that require exploration or backtracking. This thesis thus addresses that gap, investigating whether ToT reasoning can be distilled into a sub-1B parameter model using a one-step ToT framework, without requiring the student to replicate the lengthy and expensive process of the teacher.

1.4 Objective

The objectives of this study are as follows:

1. **1. Extract high-quality Tree of Thought traces from a capable teacher model:** The first goal is to generate reasoning data from a cheap but powerful model like DeepSeek v4 flash. We will use a Multi-Step ToT Controller to explore different paths, check for mistakes using a safe code sandbox, and backtrack when a dead end is hit, unlike using Chain-of Thought(CoT), which is basically a linear approach. This allows us to use complex "System 2" reasoning, where the model plans ahead and evaluates its own moves. After that, we will "linearize" these successful search logs into a One-Step ToT format. This creates a high-quality dataset across hundreds of puzzles.
2. **Identify the best small language models for ToT distillation:** We need to find suitable small language models (SLMs) that are efficient enough for edge devices but smart enough to learn these patterns. We will take models like SmolLM-360M and Qwen2.5-0.5B-Instruct. These tiny models are chosen because they already show a strong ability to perform well in arithmetic tasks. We will evaluate them based on how well they can imitate the teacher's deliberate thinking process and their overall reasoning accuracy on math-heavy tasks like the Game of

24.

3. Evaluate if the small model can use its new reasoning skills on unseen tasks:

A major part of this study is checking if the distilled model actually understands how to think or if it is just memorizing the training data. In order to do so, we will test the fine-tuned SLM on puzzles of the Game of 24 rank 901-1000, like other baselines not seen in training. We want to see if the student model can apply the "branch and evaluate" logic it learned to the problems it hasn't seen before. Success will be measured by how much the model's success rate improves compared to the baselines.

4. Use smart filtering and verification to make training more effective:

From the tree generated by the Teacher we can say not every reasoning path generated by a teacher is useful. Clean examples can help the model learn to pick better combinations, but they won't teach what to do when the combination is not correct. On the other hand, backtracking heavy examples only will encourage the model to pick bad combinations first, then try to fix them. So, depending on the actual number of puzzles solved, a filter must be used to find the balanced distribution of examples, and each example must be arithmetically verifiable to prevent the model from learning incorrect examples.

1.5 Methodology in Brief

This work adopts a teacher-student distillation framework to transfer the multi step Tree-of-Thought (ToT) reasoning from a capable, yet compact Large Language Model (LLM) to a Small Language Model (SLM). A multi-step ToT controller first generates reasoning traces through tree search, combining model proposals, code execution-based verification, and LLM state evaluations. The operation is conducted using beam search, where at each depth only top-k children are expanded while dead-end memory actively prunes/adds insight to the evaluation of states by matching patterns seen before during tree generation. The success traces are then abstracted or linearised into a compact one-step ToT and filtered based on correctness arithmetically and in terms of format. These traces are then used to finetune SLMs, and are evaluated against Chain-of-Thought and one-step ToT baselines for success rate and efficiency. Data was collected via scraping from 4games, and the raw dataset will be generated using our controller into a complete tree, which will be processed as mentioned in the pipeline.

1.6 Scopes and Challenges

This study is targeting structured, verifiable multi-step reasoning, using the Game of 24 as the primary benchmark. Game of 24 was chosen primarily due to it being a bounded problem with a limited solution space, and its results and intermediate steps are verifiable programmatically and arithmetically. Every teacher trace can be filtered based on correctness in solution and format, without any human judgment, and the distillation pipeline will only use the filtered data. For now, open-ended tasks like creative writing or commonsense reasoning cannot be verified using programmatic verifiers, so they fall outside this scope.

The dataset is generated from the game of 24 puzzles ranked 1–500 by average

human completion time, and for evaluation, ranks 901–1000 are set following [9]. The student model is restricted to sub-1B parameters for edge deployability and meeting SLM criteria. The reasoning format is restricted to a one-step format, as multi-step ToT is impossible in single inference distillation.

The key challenges encountered were as follows:

1. Path quality highly depends on the teacher model chosen. A comparatively weaker teacher can perform well using prompt engineering, but that may limit the creativity of proposals.
2. Depending on the number of unique puzzles, the ratio of examples to include in the dataset is crucial. Too many backtracks, and the model will learn to produce bad paths first, and will try backtracking till it hits the token budget. Too few backtracking examples, and the model will simply learn to give up on the first failure and learn poor backtracking signals.
3. Training on only solvable puzzles leads to the model not learning when to stop. When a puzzle is not solvable, or tried backtracking too many times, only solved puzzles don't teach it when to stop, as the model expects to find a solution no matter what. This led to the incessant backtracking seen in a number of failed cases.
4. Not all sub-1B parameter models are capable of learning the target format. SmoLLM-360M lacked vast mathematical pretraining priors and exhibited constant arithmetic hallucination that data volume alone couldn't solve. This required switching to Qwen2.5-0.5B-Instruct as the student model, which had far better mathematics pretraining, as well as being already tuned once for instruction following.

Chapter 2

Literature Review

2.1 Preliminaries

In the recent times, research in reasoning language models has improved. Initially, the ability to generate high-quality texts, answer various questions, or process NLP tasks were the major tasks for language models. However, the more complex problems started to appear in LLMs, the more important it became to generate reasons behind their answers as well. Many complex problems require breakdowns in their answers, keeping track of the previous states and comparing all possible answers before giving a final answer.

LLMs can generate long reasoning chains and follow instructions properly as they have large parameter scale and powerful instruction-following capability. SLMs have small parameter scale and do not have the same capability as the LLMs, but SLMs are more practical for deployment as they cost less and are faster at the same time. This makes it a problem to research: how to make a small student model learn the reasoning capability from a large teacher model without making the small student model expensive?

Knowledge distillation is one of the fundamental and earliest ideas to make the student models as capable as the teacher models. Hinton et al. said that a student model learns final answer generation and softer output distribution, both from a teacher model [1]. This learning method is effective because it shows how it orders different answers and gives the student model a proper indication of learning rather than only giving right or wrong answers.

DistilBERT later used this method in transformer-based language models and found that a smaller model keeps almost the full performance of a large model while being faster and lighter [2]. This method made distillation an important direction for NLP systems. However, reasoning is difficult to distill because it requires the model to maintain initial steps, make decisions across multiple states, and avoid contradictions. SLMs struggle in this case.

A major weakness of linear reasoning is that it cannot backtrack and does not support it without any structural changes. The model continues to move forward in a single path if it starts exploring that path without rethinking if there are any other possible paths or not. In tasks like solving a Game of 24, there can be multiple orders to solve before finding the correct one, and the model needs to perform different operations. A Tree-of-Thoughts model can generate and evaluate different possible reasoning ways [9]. Long’s LLM-guided ToT work also says that

the language models can work as guides inside a structured search process [7]. To solve problems that require explorations and then correction, ToT reasoning can be more efficient than CoT.

The major hindrance is that ToT requires expensive calculations. It usually generates and evaluates the generated answers multiple times before giving the final answer, which may not be very ideal for small models. One-Step ToT knowledge distillation can be very useful to solve this problem. Tree search and other complex operations will be done by using a powerful teacher model and converting the final compact one-step trace for the student model to learn [11]. Our thesis also follows this idea, as our main goal is to generate teacher traces to train smaller models. Generated data can be used as instructor data to improve instruction-following behavior [8]. However, our work does not align with this idea, as in our work, generated data does not work as work instruction data but rather as structured reasoning data.

2.2 Review of Existing Research

2.2.1 From Linear Reasoning to Tree-Based Reasoning

Chain-of-Thought prompting was introduced to increase the efficiency of reasoning by making the model write initial steps before answering the final response [4]. This technique helped large models perform better on specific tasks, for example, arithmetic and symbolic reasoning. Zero-shot CoT showed that simple instructions such as “Let’s think step by step” can show reasoning behavior in large models without giving any examples [3]. When we divided a difficult problem into smaller subproblems, prompting results improved from least to most [10]. These approaches show that even if prompting can improve reasoning, they remain linear, which we want to improve.

Chain-of-Thought (CoT) prompting has improved the reasoning capability in language models. Wei et al. introduced that models perform better on arithmetic, symbolic, and commonsense reasoning tasks when those models are asked to provide their reasoning before giving a final answer [4]. Kojima et al. also explained that large models can reason in a zero-shot setting when prompted with an easier command such as “Let’s think step by step” [3]. Still, linearity is CoT’s weakest point, and because of that, it does not fully support branching, comparing alternatives, or backtracking. This limitation is significant to implement such techniques. A single reasoning way is maintained by such models from the beginning till the end. If the model makes a wrong decision initially, without thinking or trying to understand whether that path is correct or not, it continues to follow the wrong path. Least-to-most prompting improves this by dividing complex problems into smaller subproblems [10].

Deliberate search outperforms standard prompting and CoT, and it is found in the primary ToT framework, in specific tasks that require planning and exploration [9]. Such models may make arithmetic errors or generate results that are difficult to parse. How text-only ToT works is not ideal for smaller or cost-efficient models, as its correctness depends on its ability to generate primary states, evaluate those states, and maintain an identical output format. Such models are not effective if they are asked to solve complex problems, for example, Game of 24, which requires arithmetic operations and state evaluation and may make errors or generate results

that are difficult to parse.

The main problem with ToT is it is expensive. As it goes through multiple branches to search, it requires more API calls, more tokens, and more intermediate storage than CoT. It does not align with the main idea of SLMs because SLMs are efficient and less costly. One-step knowledge distillation was introduced to reduce the gap between LLMs and SLMs by asking the teacher model to execute expensive searches and converting the final answer for the student model [11].

2.2.2 Knowledge Distillation, Small Models, and Agentic Behavior

Knowledge distillation is one of the main and earliest ideas to make the student models as powerful as the teacher models. Hinton et al. said that a student model learns final answer generation and softer output distribution, both from a teacher model [1]. This learning method is effective because it shows how it orders different answers and gives the student model a proper indication of learning rather than only giving right or wrong answers. Sanh et al. introduced DistilBERT, which is a smaller and faster version of BERT. It showed a stronger performance closer to the original larger version while being smaller and reducing inference cost [2]. Recently it has been found in research on transformer distillation that compression is important to make language models practical and useful. Brown et al. also shared a similar understanding of distillation and its importance to balance performance and computational efficiency [5].

However,

2.2.3 Code-Grounded Verification and Search Reliability

CodeAct helps reduce this problem as it allows model actions to be expressed as executable code [12]. The model can propose actions; the system can execute or verify the proposed actions instead of relying on natural language reasoning. As Game of 24 requires arithmetic operations in every step, and every arithmetic operation has a meaning; this method is very useful. The system can track if the given output by the model is correct or not and instruct the model whether to go to the next state or if it correctly follows the previous states.

Code execution as Grounded supervision supports this approach [13]. As the program execution is deterministic, it can provide reliable supervision for reasoning the steps. This is important for our thesis because the traces produced by the teacher model are mainly for training student models. If there are errors in the traces that the teacher generated, the student model may learn wrong reasoning patterns. By grounding each step in execution, the teacher model can generate cleaner and consistent traces.

Our model’s reasoning capability is not replaced by code execution in our work. Arithmetic operations and evaluating intermediate states are still the proposed by our model. Code execution layer is used as a verification layer in our work. It reduces the risk of invalid calculations or inconsistent state transitions and involves itself in planning.

Agent distillation also aligns with the same idea. Kang et al. showed that the smaller models can improve themselves from learning agent-like behavior such as

retrieval and code execution [14]. This tells us that distillation should focus on final-answer generation and be able to transfer structured behavior and correction based on constant checking. Our teacher model follows the same idea as it generates proposals, verifies calculations, evaluates states, prunes branches, and holds search traces.

It is important to make sure that the teacher model generates quality traces. The more inconsistent or invalid the teacher-generated traces are, the more error the student model may make. Our current study shows the answers or reasoning for a Game of 24 not only depends on formatted model output; it checks if the solution correctness follows the states and separately detects cases where the solution is mathematically correct but the output format is incomplete. This behavior is also important because a small model can sometimes produce useful reasoning but may not follow strict formatting requirements.

2.2.4 Memory-Augmented Search and Tree Trace Generation

ToT performs repeated reasoning. Due to various operational orders, one state may appear multiple times. The system may evaluate incorrect states if the system does not track all the states. Inconsistent output generation may appear if the system cannot keep track of all the states. If the system shows such behavior, it may become more expensive. A similar problem in lengthy conversations is identified in Think-in-Memory, as the system keeps thoughts rather than keeping reasoning over the same history in its memory [6]. To improve searching ability, the primary idea of TiM can be used: if the system knows a state is not useful, it should recall the conclusion rather than rediscovering it.

A similar concept, Dead-End Memory, applies this idea to Game of 24. It keeps track of all the states, correct or incorrect; solvable or unsolvable patterns; and productive or unproductive branches. If the system detects a normalized state, it skips it without calling the model. It helps to search more efficiently and more consistently. TiM stores conversation thoughts, and our system memory stores negative search knowledge, to be exact, which states to be skipped.

Our evaluation report shows Dead-End Memory is not only a conceptual addition; it is also used during search and contributes to reducing repeated exploration. The report also tracks `api_saved_two_num_cache` to show that deterministic two-number cache checks are applied to reduce redundant model calls for states that can be verified without LLM exploration. Tree-Structured Language Models can also become relevant to further improve this work [15].

2.3 Summary of Key Findings

Our current framework consists of a teacher model to generate reasoning trees; those trees contain useful data that can be used to train small models. Generated data are successful paths, failed paths, initial evaluations, pruning, and memory decisions.

The literature displays a visible improvement. Knowledge distillation makes smaller models more useful [1], [2], [5], but reasoning distillation is more difficult than compression. CoT and zero-shot CoT improve reasoning, but they remain linear [3],

[4]. It does not support backtracking and search [10]. ToT introduces branching search, but it is an expensive method [9]. One-step ToT tries to compress tree reasoning into a student model; its success depends on the quality of the teacher traces [11]. CodeAct and code execution improve efficient traces, which can be collected later [12], [13]. Think-in-memory and tree-structured language models suggest memory and tree traces can reduce TiM, and tree work shows that traces can reduce work for future training [6], [15].

This leaves an important gap, which is the lack of a reliable and efficient teacher-side trace generation pipeline for ToT distillation into SLMs. Existing work shows the value of ToT and distillation. But trustworthy teacher trace generation remains difficult when using smaller models. Fully text-based ToT can produce invalid calculations, inconsistent formatting, and incorrect evaluations. Our work discusses this by combining multi-step ToT search, CodeAct verification, Dead-End Memory, and consistency analysis. The target is to produce efficient reasoning traces that can be used later for SLM distillation.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

Small Language Models (SLMs) finetuned using our proposed one-step tree of thought pipeline must be able to take 4 numbers as input, to reach 24 by showing the correct arithmetic path. Instead of attempting to solve the problem directly, the SLM must generate a step-by-step process, operating on two numbers in each step, and explicitly state which numbers are left in the format of 'a' OP 'b' = 'c' '(left: ...)'. When 24 couldn't be reached, the system must be able to backtrack to the last best state and reattempt. The final line must contain the exact numbers inputted and the valid order of operations that reach 24.

Considering the baselines established in the paper [11], SLMs finetuned using the proposed pipeline must at the very least score higher than their SLM's best (9%). Moreover, the finetuned SLM should also provide a comparable success rate against their teacher LLM (GPT4o) using the one-step prompting. For a model to qualify as an SLM, it must have sub-1B parameters to fulfill our criteria of fitting inside edge devices like laptops, and the inference latency must be faster than that of APIs. The context window, including the one-step prompt and the completion, must also fit within a reasonable 1024 token limit per problem.

We used the same game of 24 dataset used by the papers [11][9], which was scraped from 4nums.com. The puzzles are ranked by difficulty in terms of average human solve time, and ranks 1-500 were chosen to start producing our finetune dataset as a bare minimum to produce diverse, clean, and backtracking examples; 446 puzzles were both correct and reliably extractable. To enable the SLM to learn valid backtracking capabilities with a limited number of unique puzzles, the linearizer script was designed to scale 446 puzzles to 1487 examples (446 clean examples, 1041 varying-backtracking examples). During finetuning, the dataset was split into 90:10 for training and validation, and the split was puzzle-level instead of random row-level, with no leakage, to prevent the SLM from seeing a different format of the same puzzle it saw during training.

Since the pipeline revolves around knowledge distillation, for the teacher model to produce raw trees, DeepSeek API access is required. While the model is deployable on edge devices, finetuning requires access to CUDA-supported GPUs with 10GB VRAM or above. The inference device must also have libraries to run the SLM:

HuggingFace, Transformers, and Optuna for hyperparameter search. During inference, to catch inconsistencies and arithmetic inaccuracies, a checker script must be run alongside to catch errors on the fly.

One non-functional requirement that was not achievable within the scope was the handling of inputs that led to no solution gracefully, which indirectly also impacted the backtrack limit. Our current system specification assumes that all input puzzles are taken from the game of 24 dataset on 4nums, and each puzzle present in that dataset has at least one solution. A complete and ideal specification would require outputting “no solution” where no combination of four numbers can reach 24, which would also teach the model a clear signal on when to give up.

3.2 Societal Impact

Distilling the capabilities of trillion-parameter models into a lightweight, 500M-parameter model that can be run on edge devices like laptops advances access to equity in AI. More adaptation into solutions like this takes reliance off of costly cloud LLM solutions like GPT or DeepSeek and helps lower the barrier for institutions and researchers in low-resource environments. Open-source SLMs that are finetuned to excel in multipath reasoning tasks like the game of 24 can be redistributed free of cost. This is contrary to LLMs like GPT or DeepSeek, which are sitting behind a paid API, highly dependent on token cost. Therefore, these enhanced SLMs could aid NLP researchers in developing countries.

Reliability of the outputs of AI reasoning systems is a critical societal consideration during deployment. A model that confidently generates output with wrong arithmetic, hallucinates intermediate steps, or gets stuck in infinite generation doesn’t instill user trust in AI-assisted tools, and reliance on such tools can lead users to over-reliance on incorrect output or simply abandon it. Our initial attempt at finetuning SmolLM-360M suffered from severe performance degradation during evaluation, either computing $10+11=11$ or miscalculating the backtracking steps entirely. This demonstrated that format adherence doesn’t guarantee arithmetic correctness or correct reasoning. While Qwen had much better arithmetic accuracy and better reasoning in backtracking, the failure cases were primarily due to the bias in the training data. The game of 24 dataset consists of problems that have at least one solution, and every example has a solution. So the model never learned when to stop, and thus, for some failed cases, it kept backtracking till token allocation for that problem. A proper remedy for this would be to synthetically generate a 4-digit combination that never sums to 24, such as “1 1 1 1”, and thus, for those problems, after certain backtracking attempts, the model will be taught when to stop. This problem was identified during evaluation, but it fell outside the scope of the current thesis timeline. Resolving this gap is a prerequisite for responsible deployment in any setting where a user may enter arbitrary inputs instead of chosen entries from the dataset, and is a clear direction for future work.

3.3 Environmental Impact

The core environmental contribution of our pipeline is the reduction in compute cost per inference. Each puzzle using our Multi-step ToT network required considerable api calls and token cost per problem. Our finetuned Qwen2.5-0.5B-instruct required only one prompt and a limit of 1024 tokens. At a large scale of thousands of users, this is a substantial reduction in data centre energy consumption.

Training cost is only one-time on a single consumer GPU, like RTX 5080 or Google Colab, available cloud GPUs like a single A100. This cost could be further cut down using Parameter Efficient Finetuning (PEFT) methods like LoRa. Compared to this, LLM solutions require repeated calls, which would be a rolling cost, increasing with the number of uses.

Beyond the game of 24, this distillation pipeline can be applied to other multipath reasoning tasks like Sudoku, creative writing, scheduling, etc. The combined energy saved by a single SLM inference over repeated LLM calls is significant.

3.4 Ethical Issues

The pipeline of linearizing full trees begins with the chosen teacher model producing full, raw traces for each puzzle. Due to choosing a flash tier cost-efficient LLM with its thinking mode disabled, as well as additional overheads introduced by our features, the teacher was guided on how to produce proposals and evaluate them. The quality of the resulting traces thus depends on how the teacher performed. Since this is a knowledge distillation, the SLM will inherit whatever strengths and biases the LLM teacher exhibited during trace generation. Various kinds of checkers within and outside the teacher, within the linearization pipeline, were introduced, which filtered out all incorrect trees, which mitigates the integrity problem but doesn't eliminate the question of diversity.

To keep the benchmark fair, 901-1000 rank problems were excluded from training. Training data was collected from puzzles ranked 1-500, which were the easier ones. Since 901-1000 problems are one of the hardest sections of the problem, training on fairly easy ones and then evaluating on harder puzzles makes evaluation conservative and honest rather than inflated.

The finetuned SLM performs well in the Game of 24 problem, but it must not be interpreted as it can solve other arithmetic problems similar to it. All code and training data are available in our repository [16]

3.5 Standards

Our evaluation strategy follows the same benchmark protocol established by Yao et al.[9]. We used the same dataset, the same puzzle rank 901-1000, and the same success rate criteria. Following this protocol ensures that our results are directly comparable to prior works.

Due to our teacher structure, the teacher can use code to automatically discard incorrect proposals. Post raw tree generation, a report script can generate aggregate and individual reports on all trees based on arithmetic correctness or validity. correct formatting and consistent metadata. Additionally, the linearizer script also

checks each tree’s correctness as well as correct formatting to convert full trees to a linear one-step format. SLM inference script also checks for arithmetic accuracy, hallucination and clear final step extraction. All methods are far more rigorous than simply checking if the final result is 24. The fine-tuning follows standard causal language modelling practices. The prompt tokens are masked with -100 in labels, and padding tokens are also masked. The training dataset is split into training/validation on puzzle-level instead of randomly to prevent data leakage.

3.6 Project Management Plan

Our 3-stage pipeline was divided into a 4-phase timeline 1. Implement a CodeAct teacher model with DeepSeek v4 flash(thinking mode off) and prompt-engineer to raise accuracy. 2. Based on the structure of the raw tree, design and implementation of the enhanced One-Step ToT pipeline 3. Select the best SLM for the task, and fine-tune it after hyperparameter optimization 4. Evaluate the test set, debug minor bugs and write a report. Since a flash tier teacher LLM was chosen, to meet previous benchmarks while also undertaking additional tasks, 2/3rd of our timeline was spent optimizing the framework, which included prompt engineering to ensure proposal coverage and fast evaluation. Once the structure of the raw trees from the teacher can be fixed, work can begin on the specifics of linearizing the trees. Fine-tuning the SLM was the last step and takes the least time. With the current DeepSeek v4 flash non-thinking token cost (\$0.14 per million input tokens, \$0.28 per million output tokens, \$0.0028 per million input tokens when cache hit), the budget must be enough to cover at least 600 puzzles (500 training puzzles, 100 test puzzles). Access to Google Colab A100 or a local consumer-grade NVIDIA GPU with at least 12GB VRAM for fine-tuning the SLM. Depending on the result of the fine-tuned SLM, further adjustments were applied to the augmented data to enhance SLM learning capabilities. After testing SmolLM-360M was swapped with Qwen2.5-0.5B-instruct to combat arithmetic hallucination, and training data was enhanced from the preliminary version, adding a better backtracking approach. While choosing SLMs for finetuning, SLMs with better mathematical pretraining were prioritized. As a result, choosing Qwen2.5-0.5B-instruct netted a greater success rate than [11].

3.7 Risk Management

Developing a knowledge distillation pipeline for SLMs introduces a range of risks in technical, methodological and resource-related sections. For our pipeline mitigation were applied after identification, but some limitations remain unresolved within our current scope.

Not all models have the capacity to handle the target task. The initial candidate SmolLM-360M chosen in alignment with the baseline of [11] exhibited severe arithmetic hallucination, which was primarily due to its lack of mathematics in its pretraining corpus. Our pipeline can teach the model how to reach the goal and how to backtrack, but its inability to perform simple arithmetic, as well as its bad backtracking capability, was difficult to mitigate with just data volume alone. The risk was mitigated by switching the student model to Qwen2.5-0.5B-Instruct. Qwen2.5-0.5B-Instruct has a stronger mathematical pretraining, further adjusted

by the RLHF instruction finetuning. All subsequent training, hyperparameter optimization, and evaluation were conducted exclusively on Qwen2.5-0.5B-Instruct.

Due to working on a small section of the puzzle (446 unique puzzles), which was further expanded into 1487 clean and backtracking examples, there is a risk of the model overfitting and memorizing the puzzles instead of learning patterns. This was mitigated by tuning hyperparameters and training with early stopping.

Incessant backtracking was seen at inference, where the model, upon not finding a solution immediately, tries to backtrack endlessly till token limit is hit. After studying the behaviour as well as the dataset distribution, it was concluded that this primarily happened due to our teacher never seeing a problem that has actually no solution, hence the fine-tuned student also adopted this behaviour. This problem and relevant insight were identified during inference, but couldn't be addressed within the project timeline.

Further fine-tuning an already fine-tuned Instruct pretrained model is highly sensitive to hyperparameter tuning, as larger values can cause the model to forget the previous fine-tuning weights, and this problem is known as catastrophic forgetting. To mitigate this, upon multiple tests, very conservative and smaller hyperparameters (details were explained in later sections) were chosen to add the game of 24 solving as a skill on top of its instruction following capabilities.

3.8 Economic Analysis

For generating raw trees using the teacher model, each puzzle would produce trees that would require a significant amount of api calls and token cost. Since all fine-tuning was done on local hardware, the only additional cost incurred was compute time. Each trial/final fine-tuning process on an RTX5080 for 5 epochs with early stopping took less than 20 minutes.

Once fine-tuned, the resulting Qwen2.5-0.5B can run locally without any ongoing API cost. Thus, after the one-time cost of producing training examples, the fine-tuned SLM will pay off its training cost after a certain use, and from then on will run for free indefinitely.

While we chose Gameof24 as a benchmark, this pipeline generalizes to any structured multipath reasoning task that can be verified programmatically, and can be further enhanced to work without that, too. The one-time distillation cost produces a reusable dataset that eliminates future per-query costs.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

The main problem our work addresses is straightforward, which is that large language models are very good at solving hard reasoning puzzles, but they are slow and expensive. This is where we focused on small models as they are fast and cheap, but they struggle with hard problems as they require multi-step thinking. The goal of this work is if we can manage to transfer the reasoning ability of a large model to a small one through a process of knowledge distillation.

For our test case, we use the Game of 24 and the rule is that there will be four numbers, and the task is to combine them using simple arithmetics like $+$, $-$, $\times$ and $\div$ in order to reach 24 using each number only once. This is considered a good benchmark for reasoning because it asks for genuine search in order to get to the results, as it requires trying a path, recognizing it fails, then going back and trying another path rather than simply recalling a fact.

The Tree of Thought idea is the backbone of this entire project, as every stage either produces a tree, converts a tree or learns to mimic one. Before describing the three stages of the entire pipeline, it is worth explaining why we have shifted from Chain of Thoughts (CoT) to Tree of Thoughts (ToT).

The shifting: From a chain to a tree. The most common way of reasoning is Chain of Thought (CoT), in which the model reasons in a single straight line from start to end, and it works pretty well in problems whose path to the answer is direct. But they struggle in search problems like the Game of 24. To solve these problems we need to explore more paths, so it requires backtracking, but the straight line cannot go back. Once CoT commits to a move, it gets locked in it. If the first move fails, as the probability is higher as there can be multiple paths to solve 24, the entire attempt fails and there is no chance of reconsideration. In these cases, everything the model tried along the way to solve the problem is simply discarded, leaving no trace.

Here comes Tree of Thoughts (ToT) as a savior as it removes both limitations. Instead of one path, it explores many paths at once, and at each step it proposes several candidate moves and scores how promising each of the nodes look, accordingly keeps the best few and abandons the rest. When the model reaches a dead

end it then goes back to where it started and takes another path, which is like an algorithmic version of a person trying to find a path that will work and, releasing when it does not work, simply goes back to try another. This ability to **look ahead and backtrack** is the exact quality that hard problems require to get solved, and this is where CoT fundamentally lacks.

Why the tree matters for distillation, not just for solving. There is an-

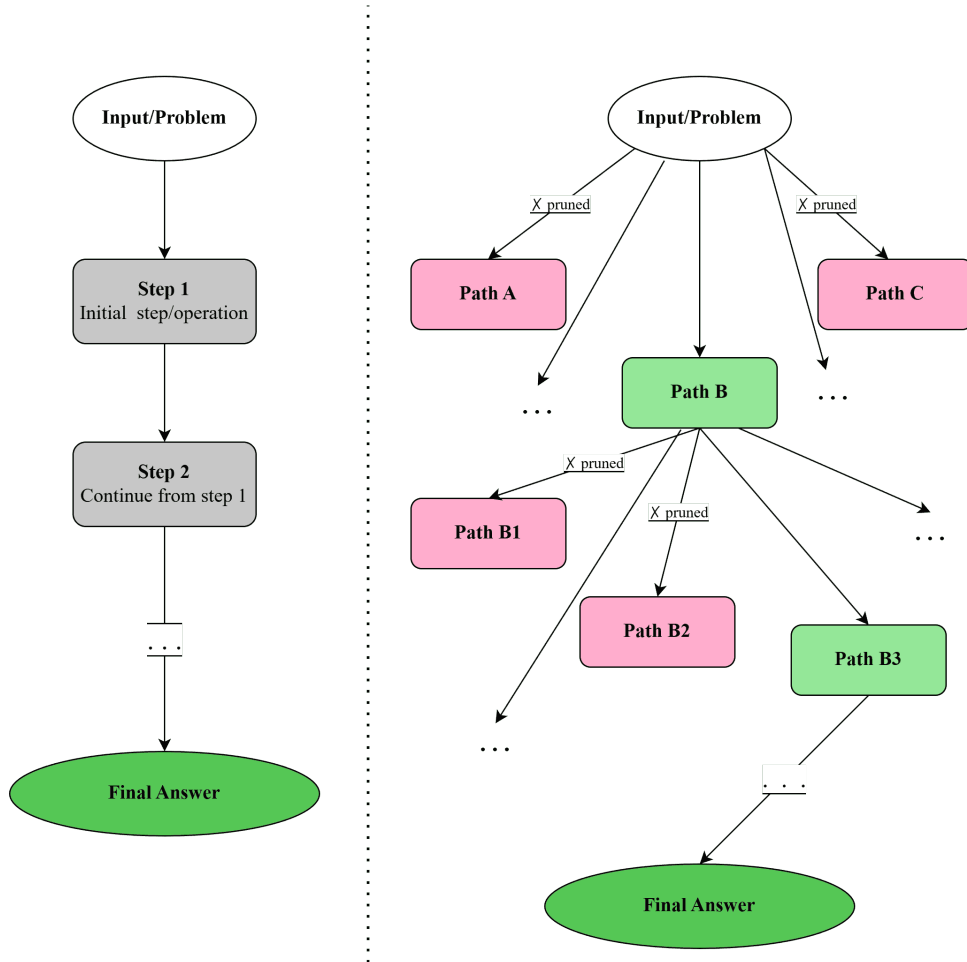


Figure 4.1: CoT vs ToT.

other reason equally important to use a tree: as the search runs, the tree records everything — not only the successful paths that eventually reach 24 but also the paths that have failed to do so. In CoT there is no record of the paths that fail. In ToT, they are stored as part of the tree because the ultimate goal is to teach and make it easy for the small models to recognize the dead ends and recover from them, so those recorded failures are not a waste; they are some of the most valuable training materials we have. The tree is effectively a complete record of how to think through a problem, taking notes on mistakes, which is basically how humans try to solve a mathematical problem.

The methodology has three stages and ToT runs through all of them.

- **Stage 1: Teacher (Multi-step ToT).** A large model (Deepseek v4 flash, non-thinking mode) tries to solve each puzzle with a full multi-step tree search.

It keeps records of the successful path and failed branches of the entire tree as the raw reasoning data.

- **Stage 2: One-step ToT Conversion.** Each multi-step tree is flattened into a single linear path that carries all the essential behaviour of the tree, such as try a path, hit a dead end, backtrack and continue. This simple “one-step” format helps the small model to actually learn.
- **Student.** A small model (Qwen 2.5 0.5B-Instruct) is fine-tuned to reproduce that one-step ToT behaviour so that it can imitate the teacher’s tree search in a single generation — all without any actual tree and no search at inference time.



Figure 4.2: Simplified pipeline.

In short, the project is fundamentally about compressing a multi-step tree search into a single forward pass. In this process, the expensive, deliberate ToT reasoning of a large model is captured as data and then taught to a model small enough to run on any device.

How our work is different from the closest previous work. The most similar published work by Renduchintala et al. [11] also distills one-step ToT into a small model but it does not build a real tree to generate data. Their training examples came from a single prompt on GPT-4o which only imitated ToT-style reasoning and scored 19%. But we run a genuine multi-step tree search that scored 89% and flatten those trees into training data. The quality and richness of the data is the primary difference from their work.

4.2 Preliminary Design or Design (Model) Specification

4.2.1 Stage 1: The Teacher (A Multi-Step Tree of Thought Solver)

At its heart, the teacher is a large language model, specifically **Deepseek v4 flash (non-thinking mode)**, that for each puzzle builds a Tree of Thought (ToT), which systematically searches through possible moves and stores the paths that successfully reach the solution as well as the ones that failed. The word “non-thinking” means the model does not do any hidden internal reasoning of its own before answering; it simply responds to what it is asked. This is a crucial point. We chose this model on purpose because we wanted the careful, deliberate reasoning to come from the search-based information of the tree — the scoring and the backtracking. In clear words, the intelligence of our tree comes from the Tree-of-Thought, not from

the model’s black box.

One principle runs the whole design process. The language model is deliberately asked to do two things that it is good at: proposing suitable candidate moves and evaluating (judging) how promising each move is. Every other operation — such as arithmetic, pruning, and selection — is performed by deterministic Python code. Figure 4.3 shows the complete workflow of the process.

Following the standard formulation of Tree of Thoughts (ToT), every tree search of this kind falls into these four questions: (1) How to break the problem into thought steps, (2) How to generate suitable candidate thoughts, (3) How to evaluate each state/node, (4) Which search algorithm to use? Our teacher answers all four questions.

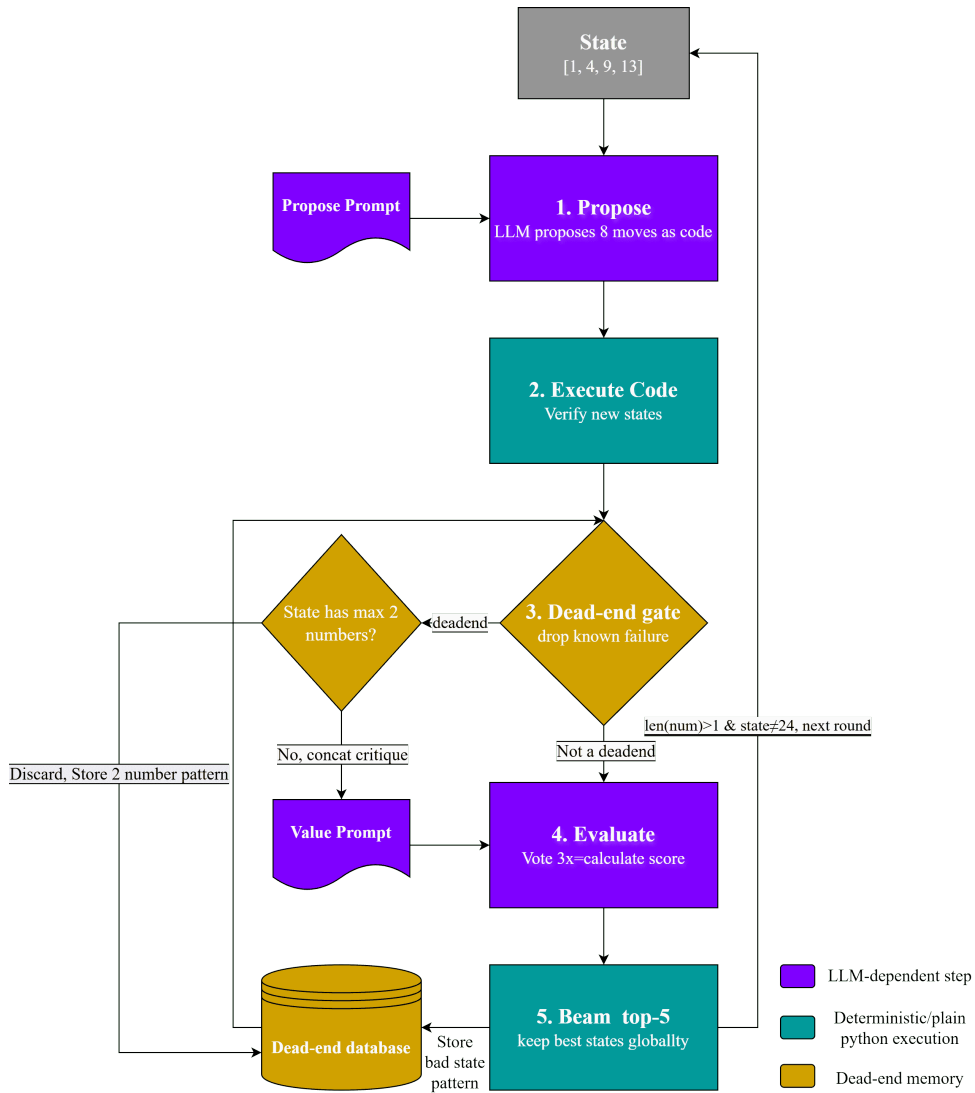


Figure 4.3: One round of the teacher search.

Alternative designs for the dead-end memory. The most complex design decision in the teacher was the level of aggressiveness in pruning. Three versions were built and compared, each answering the question differently. The REVERT version applied the lightest pruning and the model saw most of the tree, so its

reasoning stayed closest to human-like. In this design, the dead-end memory is barely used because it only skips a node when the two-number pair is impossible to result in 24. The major drawback was that it is the slowest and most expensive as the token count increases. The 3D-DEM version is quite aggressive in pruning: when a three-number state matches a previously failed number set it immediately discards it before the LLM is called, so the model never sees that number set. This approach is fast and effective but made the search less authentic. The PATCH version keeps the hard skip only for the deterministic two-number cases where a wrong answer is mathematically impossible; for three-number sets it does not discard immediately but rather adds a warning hint to the evaluation record and still lets the LLM evaluate the state. This is the final version used to generate all 500 raw trees.

Three Alternative Teacher Design(Dead end memory trade off)

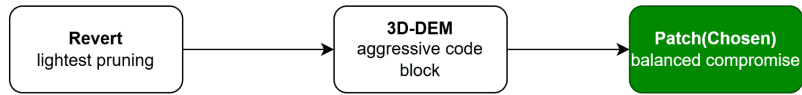


Figure 4.4: The three alternative teacher designs.

Figure 4.5 shows how a real search tree works and finds its path. The example “1 2 4 7” is taken from our raw tree generated by the teacher. The core architecture for Tree of Thoughts in Game of 24 breaks the problem into steps, where at each step two numbers produce one number by a simple arithmetic operation, reducing the set until a single number remains. This entire process builds a tree where:

- **Root Node:** the starting state, e.g. [1, 2, 4, 7]
- **Each child node:** the result of one arithmetic step, e.g. “add 1 and 2 to get 3, now I have [3, 4, 7]”
- **Leaf node:** either a solution (24) or a dead-end

From the answers to the four questions, the Tree of Thought framework specifies how thoughts are decomposed, generated, evaluated, and searched with the specific design we made in our project.

- **Thought Decomposition:** A thought is a single arithmetic move that combines two of the remaining numbers into one, reducing the count by one: four numbers becoming three, then two, then one (the final result). A complete solution is a path of exactly three thoughts, meaning the depth of the tree is three.
- **Thought Generation — the Proposer (CodeAct):** At every node, the proposer generates approximately eight possible combinations of the next move from the current number set. Each proposal is written as executable Python code, which is then run to obtain the result. The output is always taken from the code (e.g. `res = numbers[3] - numbers[2]`), not from the model’s

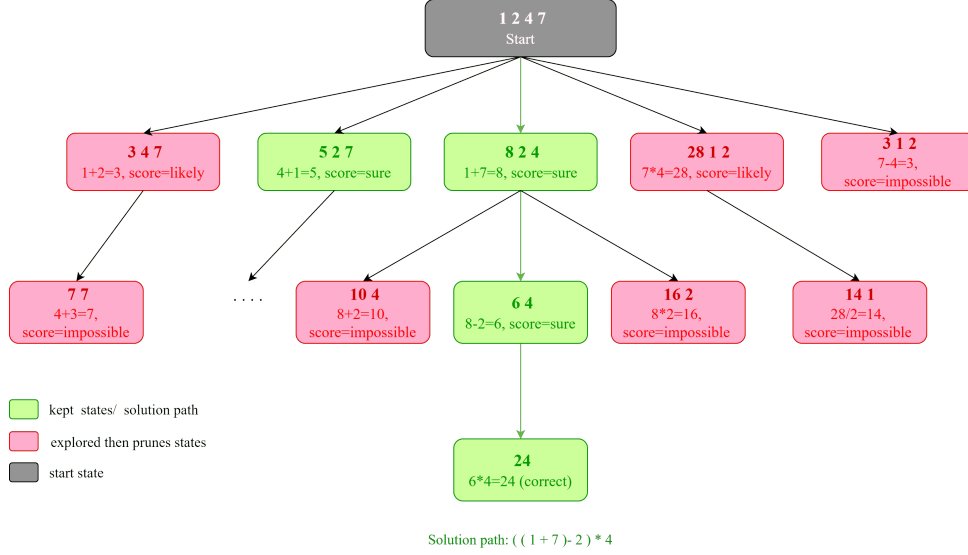


Figure 4.5: A real raw search tree from our teacher for the puzzle “1 2 4 7”.

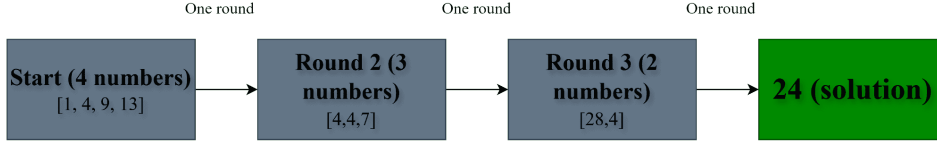


Figure 4.6: Thought Decomposition.

prose. This eliminates two failures at once: arithmetic hallucinations and text-parsing ambiguity. The proposer runs at a temperature of 0.7 so that the search explores different branches rather than repeating the same moves.

- State Evaluation (the Evaluator and the Scoring Tier):** Each proposed state is then scored to see how promising it is. Trivial cases are resolved deterministically: a single number equal to 24 scores 1.0 (sure); any other single number scores 0.001 (finished and failed); two numbers giving a premature 24 score 0.01; and any state with a value greater than 1000 scores 0.1. For everything in between, the model is called three times at a temperature of 0.0 and the majority score is accepted. The model reasons backward from the goal and returns “Sure” = 1.0, “Likely” = 0.6, or “Impossible” = 0.001. This LM-based evaluation is the heuristic that controls the whole search, and the score is stored on each node so that the beam can read it without any overhead.
- Search Algorithm (Global Beam Search with Dead-End Memory):** The tree is explored breadth-first, processing all nodes at one depth before proceeding to the next. At each depth, all proposed states from all surviving nodes are pooled into one list, sorted by score, and only the best five are kept (beam width = 5). This global pooling means a weak parent cannot waste a beam slot if a strong child from another parent scores higher. Before the costly evaluation step, the **dead-end memory** is checked so that matching states are discarded without any LLM call. The gate checks two points: (1) whether

the state has been seen before — if so, it is dropped immediately; (2) for two-number states, a deterministic check runs all four operations in code and discards the pair if none reaches 24. Three-number states are not blocked even if seen before; they are passed to the evaluator with a soft warning appended. The memory is shared across every puzzle and helps the teacher to solve more puzzles while making the search more efficient.

4.2.2 Stage 2: One-Step ToT Conversion Design

A multi-step approach is rich but also too complex for a small model to consume. This stage flattens each tree into a single linear sequence that preserves the tree’s search behaviour — trying, failing, and backtracking — in a simpler form. The design has three parts: extraction, augmentation, and verification.

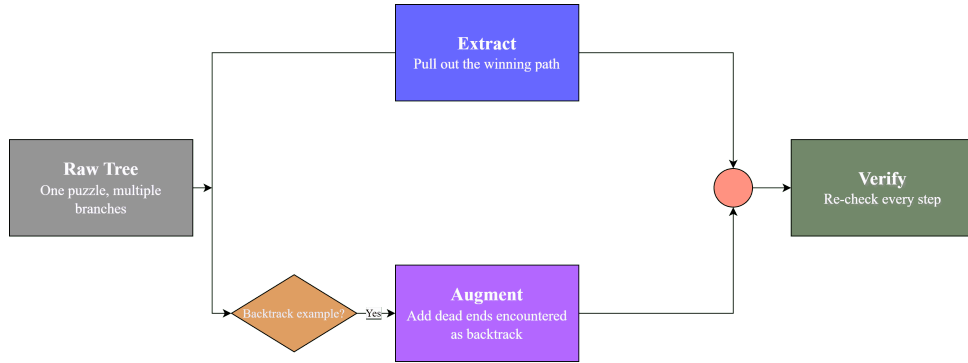


Figure 4.7: One-step data augmentation pipeline.

- **Extraction and the Pool-Based Expression Builder.** The winning path is traced from the solution leaf back to the root and then written up step by step. Each step is read from the executed Python output rather than the model’s own words. The final answer is reconstructed as one nested formula, such as $(7 \times (13 - 9)) - 4 = 24$, and a pool of (value, expression) slots is used rather than a dictionary keyed by value, because two different sub-expressions can produce the same number. For example, for puzzle $[1, 1, 2, 6]$:

- Step 1: $1 + 1 = 2 \rightarrow$ pool slot: (2, "1+1")
- Step 2: $6 - 4 = 2 \rightarrow$ pool slot: (2, "6-4")

A plain dictionary would silently override one of the 2s; the pool keeps duplicates in separate slots and consumes each exactly once, enforcing the rule that every number is used exactly once.

- **Alternative Designs for Augmentation (v1 vs v2).** After extracting the winning path, dead-end memory from the teacher is spliced in before the solution so that the small model learns about failures as well as successes. Two versions were built to teach the student to recover from dead ends. Version 1 always backtracks to the root when it faces a dead end, which is not realistic and teaches the student an “always restart” habit. Version 2, the chosen design, targets the actual branching point where the failed branch diverged from the solution — sometimes the root, sometimes an intermediate state.

This version produces 632 non-root backtracks within 1,487 training examples, giving roughly 30% examples with no backtracks, 30% with one, 25% with two, and 15% with three or more.

- **Functional Verification of the Data.** An independent checker re-reads every generated example and confirms eight properties, including that each arithmetic step is correct, that pool states match after each step, that no new number appears out of nowhere, and that the final formula evaluates to exactly 24. It also uses an independent brute-force solver to verify that dead ends are genuinely unsolvable. Any backtrack from a winnable position is flagged, as teaching a student to abandon a winnable state would be harmful.

4.2.3 Stage 3: Student Training

Alternative designs (two student models)

The 1487 verified examples were used to train two student models, and the final model chosen was primarily based on design choices. SmolLM-360M, a 360M-parameter model, was chosen at first despite its being a base model with limited mathematical training to match the closest prior work. However, upon SmolLM’s failure (see the Final Design Adjustment in Chapter 5), a 500M-parameter instruction-tuned model called Qwen2.5-0.5B-Instruct was chosen due to its strong arithmetic reasoning.

Training specification

Supervised fine-tuning with prompt masking was used to train the models. Using this approach, only the completion part is graded in the loss section; during fine-tuning, puzzles and in-context examples are shown to the model, and their tokens are masked from the loss. The dataset was split into 90% training and 10% validation, and the split was done at the puzzle level. The reasoning behind this decision was to prevent different variations of the same puzzle from getting separated in the training and validation sets, which would’ve led to inflated results during validation, thus preventing leakage. For SmolLM-360M, hyperparameters were found by using an Optuna search for 10 trials and 12 epochs (best: learning rate 2.57×10^{-5} , batch 4, weight decay: 0.0369, constant-with-warmup). Since Qwen2.5-0.5B-Instruct is already an instruction-tuned model, after a few manual trials, conservative hyperparameters were chosen (learning rate: 1.5×10^{-5} , cosine scheduler, fewer epochs). These hyperparameters were chosen on the principle of teaching an already instruction-tuned model to learn a new skill, and hyperparameters like the ones used for SmolLM have a large chance of overwriting its previous weight updates, called catastrophic forgetting.

Tokenization specification and a corrected design flaw

To fit the roughly 280-token in-context header, the puzzle, and the completion, the context window is set to 1024 tokens. One subtle but critical specification used was the truncation direction, where the prompt was never truncated, but was trimmed from the right if the token count exceeded the maximum. The first version of the implementation had the crucial mistake of truncating from the left, which trimmed the prompt and mis-masked the completion as well. As a result, the preliminary fine-tuned model was trained on no new labels, which resulted in the generation of

garbage. This direction was corrected in the next versions. Further, the context window was raised, and label sanity checks were added to prevent any design flaws before the student can learn.

Chapter 5

Result Analysis

5.1 Performance Evaluation

In our work all evaluation uses the same test set: puzzles ranked 901–1000, considered hard problems on 4nums.com by difficulty scale. A puzzle is only considered solved if the model produces a formula which evaluates to exactly 24 using each of the four numbers only once.

Teacher results: The teacher solved 89 trees out of 100 test trees with zero arithmetic errors and zero format errors, confirmed by an independent state-chain verifier. On the training set it solved 450 trees out of 500 (puzzles 1–500), giving an approximately 90% success rate.

Student results: SmolLM-360M solved 0 out of 100 test puzzles, while Qwen2.5-0.5B-Instruct (fine-tuned) solved 24 out of 100. The untuned baseline of Qwen also scored 0, which confirms that fine-tuning genuinely added the skill.

Model	Solved out of 100	Success Rate
Teacher: Deepseek v4 flash (non-thinking mode)	89	89%
Student: Qwen2.5-0.5B (fine-tuned)	24	24%
Student: SmolLM-360M (fine-tuned)	0	0%
Qwen2.5-0.5B (no fine-tuning, baseline)	0	0%

Table 5.1: Success rate of distilled students on the same test dataset (puzzles 901–1000). Our best model Qwen performs at nearly double the success rate of the comparison result from previous work.

5.2 Analysis of Design Solutions

In the teacher’s design process each part contributed directly to the 89% success rate of tree generation. Among them the three most important are the dead-end memory, the global beam selection, and CodeAct.

Dead-end memory: Figure 5.2 gives a closer look at the 100 test trees. At depth 1 the search stays small, but at depth 2 the tree expands noticeably and branches

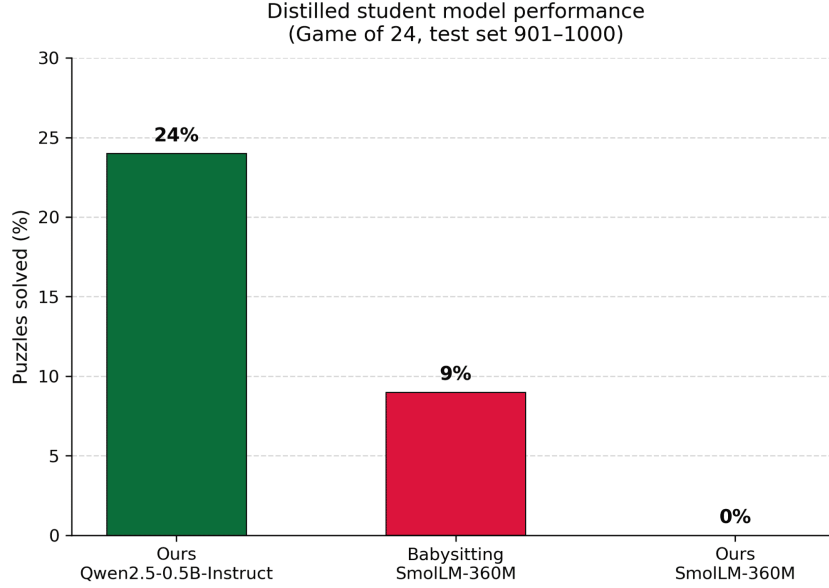


Figure 5.1: Distilled student performance on the test set for Qwen, SmolLM (their paper), and SmolLM (our work).

into more than 5,400 states. This is where the dead-end memory does its job: of the 5,405 states at depth 2, only 3,835 are actually built; the remaining 3,586 states are pruned by the dead-end memory before any LLM call is made. By depth 3 only genuinely solvable states remain. This is the single biggest efficiency gain: it makes the teacher’s search process comparatively less computationally expensive and, most importantly, makes generating 500 training trees affordable.

Global beam selection: This keeps the top five states (beam width = 5) globally rather than one winner per parent, which means the search focuses its entire budget on the most promising branches. Keeping one winner per parent would have wasted beam slots on weak parents.

CodeAct proposals: Every move is evaluated based on the result of executable Python code, which eliminates two major failure modes: (1) arithmetic hallucinations (the model proposing “ $13 - 9 = 5$ ”) and (2) text-parsing errors (trying to extract a number from ambiguous prose). Every arithmetic state is verified by Python code, not by trusting the model’s own words.

Student failure mode: The complete failure of SmolLM-360M (0%) reveals something important about the student design. Figure 5.3 shows that SmolLM-360M trained well by the usual measure: its evaluation loss fell smoothly from 0.74 to 0.16, and a low evaluation loss indicates a successful fit of the model. Yet that low loss did not help it solve a single puzzle. Although it learned the surface format of the training data — correct steps, correct backtrack style, correct arithmetic — it never learned the judgement of when to commit to a final answer. As a result it fell into a loop, backtracking repeatedly without stopping. Qwen, trained to an even lower evaluation loss (near 0.096), did learn to finish. The key takeaway is that loss alone is a poor predictor of success: a lower value confirms the model imitates the surface form of the data, not that it has acquired the reasoning to solve the task.

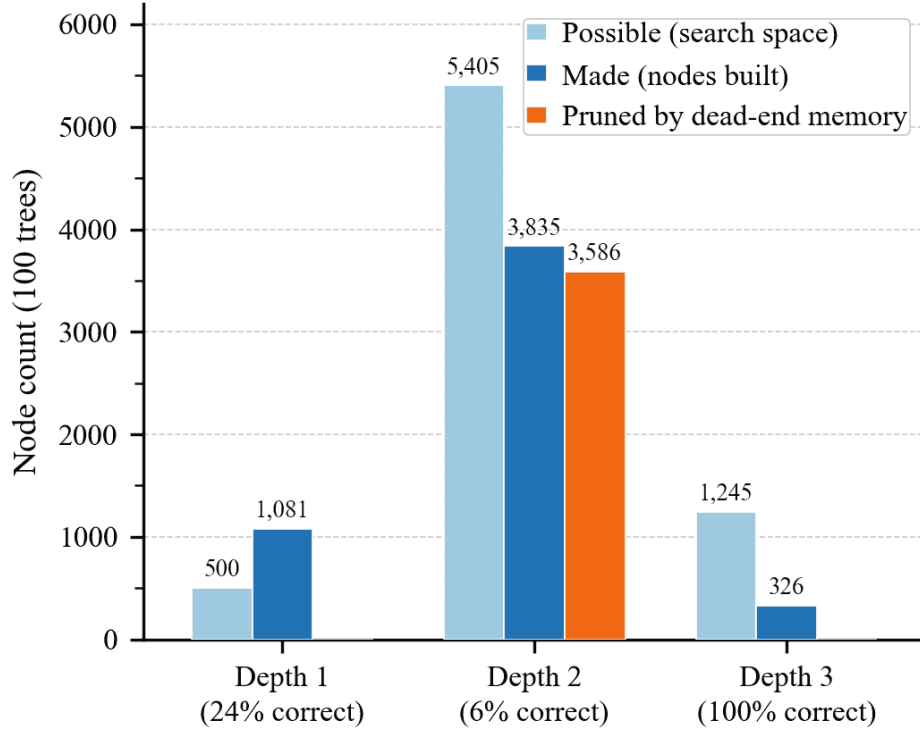


Figure 5.2: Depth expansion and pruning across 100 test trees (puzzles 901–1000).

The backtracking curriculum requires a student with enough capacity to learn not just when to backtrack but also when to stop.

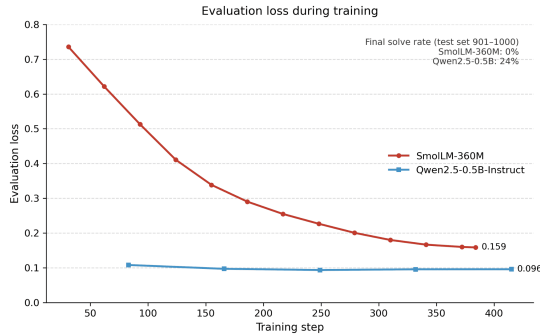


Figure 5.3: Evaluation loss during training for SmolLM-360M and Qwen2.5-0.5B-Instruct.

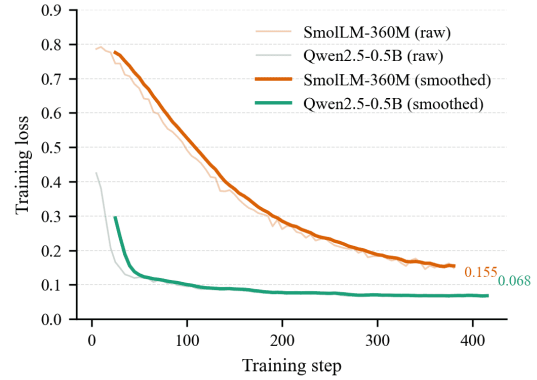


Figure 5.4: Training loss during training for SmolLM-360M and Qwen2.5-0.5B-Instruct.

Final design adjustment: After observing SmolLM-360M’s failure despite low training loss, one deliberate adjustment was made to the student stage: the base student model was replaced with Qwen2.5-0.5B-Instruct. This change does not affect the training data, the training procedure, or any part of the teacher or one-step ToT pipeline.

The reason is straightforward. SmolLM-360M is a base model with general-purpose

pretraining but limited mathematical ability. Qwen2.5-0.5B-Instruct, by contrast, had been specifically pretrained on mathematical and structural reasoning and had undergone instruction tuning, giving it a stronger sense of when a task is complete. These abilities were present in the model before any fine-tuning, which made Qwen far better placed to absorb the backtracking curriculum and produce a finished answer.

When shifting to Qwen the training settings were also eased off, since Qwen already knows how to follow instructions and already has math skills. The goal was to teach it one new trick rather than retrain it from scratch. A gentle learning rate was therefore used (1.5×10^{-5} instead of 2.57×10^{-5}), training ran for fewer epochs (5 instead of 12), and a cosine schedule let the learning rate fade smoothly toward the end rather than staying constant with a warmup. This gentle push was enough to turn 0% into 24%, as shown in Figure 5.1.

5.3 Statistical Analysis

The following tables summarise the key statistics for the teacher and the training data.

Teacher efficiency (averaged per puzzle over 100 test puzzles):

Metric	Average per puzzle
Total nodes built	53.4
Model API calls	45.2
States skipped by dead-end memory	35.9
Dead-end skip rate	70–80%
Wall-clock time	~80 s
Input tokens per puzzle	43,189
Output tokens per puzzle	25,144

Table 5.2: Teacher efficiency metrics averaged over 100 test puzzles.

Training data distribution (total 1,487 examples):

Category	Count	Share
Clean (0 backtracks)	446	30%
1 backtrack	446	30%
2 backtracks	372	25%
3 or more backtracks	223	15%
Non-root backtracks (total)	632	—

Table 5.3: Distribution of backtrack complexity across the 1,487 training examples.

The 70–80% dead-end skip rate is the most important figure here: without it the teacher would call the LLM three times for evaluation on every state. Skipping the majority of states saves a large amount of work and makes the teacher affordable to run across hundreds of puzzles.

The deliberate split of 30% / 30% / 25% / 15% was chosen to take the student from easy to harder patterns progressively, so it learns the basic structure first. At the deepest backtrack level (15%) the student must navigate several failed paths before finding one that works.

It is worth pausing to appreciate the sheer volume of training data produced by the teacher’s full tree recording. Because the teacher stores both failed attempts and successful paths, every solved puzzle yields not just the winning path but also the wrong turns alongside it. The Babysitting paper [11] did not go into these complexities; instead, they ran their teacher over the puzzles, kept only fully correct solutions, and discarded the rest, leaving them with just 144 clean examples to train on. Our approach gives 1,487 examples from 500 puzzles, whereas they extracted 144 examples from 1,362 puzzles — giving us roughly ten times as much training data from less than half the puzzles, and without running the teacher any extra time.

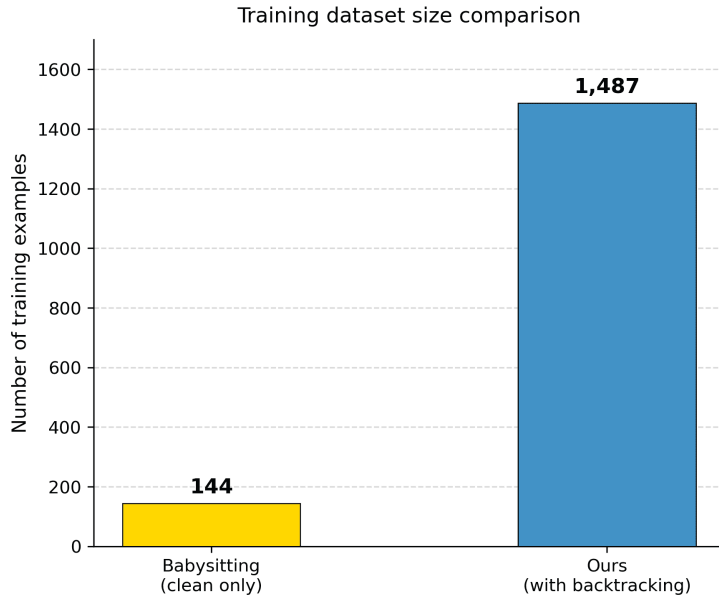


Figure 5.5: Training dataset size comparison between our work and the Babysitting paper.

5.4 Comparisons and Relationships

We compare against two reference points. The first paper is our tree-of-thoughts base paper (Yao et al., 2023)[9] from where our entire idea is of the central method of reasoning with Tree-of-thoughts(ToT) came and the other one is the comparing paper (Renduchintala et al., 2025)[11] which has done the closest related work. Both of the papers uses 901-1000 as test set of Game of 24 and both show Chain-of-Thought(CoT) baseline.

In the Babysitting paper(Renduchintala..)[11] shows two different teacher results and its important not to confuse between them. Their strongest teacher is a full

multi step Tree of Thoghts which reached 82% success rate at breadth b=5 but that is not the teacher they distilled the knowledge from due to avoid the complexity. To make distillation easier for the SLM to grab they used a teacher with a single-prompt “One-step ToT” and this teacher scored only 19%. This is the teacher they used from knowledge distillation into Small model (SmolLM-360M).

Why they distilled from weaker model even though it scored lower? It worth being clear that the babysitting paper didnt choose the “one-step ToT” as teacher because its better, they chose it according to their paper distilling from “multi-step ToT” can not be possible or rather say too complex as it has to deal with couple of things such as it is an external search algorithm(breath-first search at b=5) that calls the model three times per puzzle to propose,evaluate, and prune. Their “multi-step ToT” generates no input-output sequence for the student to imitate in one forward pass, and in order to do knowledge distillation we needed that. This was the limitation of this paper that set out to solve.

Our pipeline takes the harder path and avoids that compromise. Rather than using

Model	Rate
Teacher: One-Step ToT with GPT-4o (renduchintala et al.) [11]	19%
Student: Fine-tuned SmolLM-360M (renduchintala et al.) [11]	9%
.....	
Teacher: Deepseek v4 flash (ours)	89%
Student: Qwen2.5-0.5B-Instruct fine-tuned (ours)	24%

Table 5.4: Comparison paper results [11] (above dashed line) vs. our results (below) on test puzzles 901–1000.

a weak teacher to fit in the requirements of distillation, we keep the full multi-step search and add a **linearizer** that flattens each correct search tree with it’s winning path, its dead-ends and its backtracks into a single linear sequence.

This is the core contribution of our project: The babysitting paper tells that a real multi-step teacher cant be distilled and worked with 19% teacher model whereas we used the 89% multi step teacher and flatten it so that the student learns easily. In Figure 5.6 we can see the teacher gap which is ours 89% and theirs 19%.

In Figure 5.6 and 5.7 shows the same results we have discussed already, and the nuances are we measured them with two different reference line because two distinct questions mattered here. Figure 5.6 sets the **CoT baseline (7%)** for the weak prompting result that any reasoning method must beat to justify the strength of itself. After measuring againgest the CoT baselines we can say that our teacher beats the baseline by roughly twelve-fold and our distilled student by more than three-fold.

Whereas, Figure 5.7 draws the **competing teacher’s ceiling(19%)** and after measuring against the competing teacher’s ceiling we can state our student (24%) solves more of these hard puzzles than the entire teacher of their in comparison as their teacher scored (19%). A small 0.5B parameter student outperforming the larger model indicates that the result depends one the quality of the search tree rather than the act of distillation because our teacher records its failed branches with their

successful ones which make it learn not only the correct paths but also recognising and recovering from failed ones.

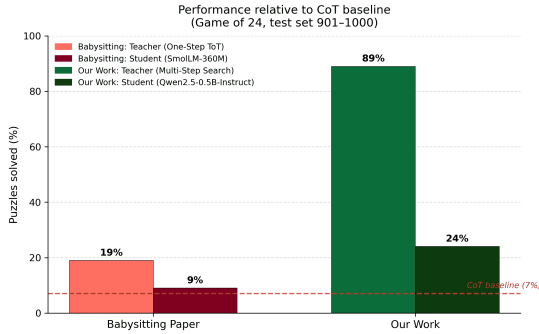


Figure 5.6: All four results against the CoT baseline (7%, dashed). Every result clears it, but our teacher (89%) and student (24%) clear it far more decisively than the babysitting teacher (19%) and student (9%).

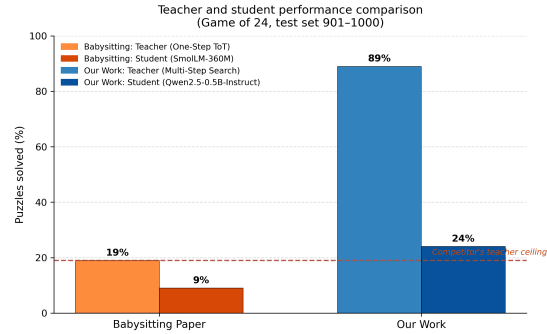


Figure 5.7: The same results against the competing teacher’s ceiling ($\approx 19\%$, dashed). Our distilled student rises above the line and it solves more than the model that generated the rival dataset.

The relationship across the pipeline. A stronger teacher (89% versus their 19%) generates data that is both larger and structurally richer (Figure 5.6 and Figure 5.7). That distilled data makes the student perform well which lifts the student’s ceiling (24% versus 9%). Each stage carries its advantages to the next step when the teacher’s tree becomes the dataset’s non-root backtracks and the dataset’s non-root backtracks become the student’s recovery skill.

5.5 Discussions

The results validate the central thesis. A richer and more carefully made teacher produces better distillation data, which can enhance a small model to demonstrate better performance on difficult reasoning problems. Our claims are supported by our teacher’s 89% solve rate with zero-error verification, and the nearly ten times more example count over the comparison work. Similarly, the results of the student can help us draw a clear line on the limitations of distillation alone. Training loss, despite looking excellent for both students, told us no story. SmolLM-360M reached a lower loss compared to Qwen2.5-0.5B, yet it solved nothing. The 0% vs 24% came from the student’s prior pre-training on mathematics, not on the training data. Our hope of generalizing teaching the student to try, fail and backtrack was the most useful finding using training examples of 632 non-root backtrack events, where the comparison paper used none. We are careful about our claim here. Our best student is different from theirs in both the base model and training data. So, we cannot claim the 24%-versus-9% gap is the result of the data alone. The controlled example of using SmolLM-360M actually resulted negatively (0% versus 9%) because our backtracking format required more capacity than the model had. The honest way of stating this is that our richer data only showed its capability once a higher-capacity model was used. Thus, the data format’s benefit and model capacity are interconnected in this experiment and not separable.

Limitations: The unsolved test puzzles for the teacher (11 puzzles) were primarily difficult cases, where the proposer failed to give a crucial first step at depth 0. To solve it, the beam width can be widened, or the proposal prompt can be further engineered without hardcoding any behaviour, or alternatively, run the same puzzle n-times. The failure cases of the best student shared the trait of incessant backtracking until the token limit was hit. This was primarily due to the game of 24 dataset, where every problem had at least one solution. Thus, our training data had problems where, with multiple backtracks, it always led to 24, and the student model also learned that whatever number is entered, it must backtrack till it hits 24, never learning to give up. This resulted in those incessant backtracks, and with each backtrack, it forgets the root beyond its context window, and more hallucinations appeared. The student’s performance remains far short of the teacher’s at 89%. To close that gap, we need a teacher who can propose more diverse and creative paths, more unique puzzles, or both. Finally, all results are on a single benchmark of the Game of 24. The generalization of other multi-step reasoning tasks wasn’t conducted in our work.

Chapter 6

Conclusion

6.1 Summary of Findings

A CodeAct version of the ree-of-Thought teacher with persistent dead-end memory solved 89% of hard Game-of-24 puzzles with zero arithmetic errors. The raw trees from the teacher were backtrack augmented, expanding to 1487 examples from 446 puzzles, which is roughly around 10 times more than the closest published work, and required running on less than half as many source puzzles. A small student fine-tuned on those examples had a gain from 0% to 24% on the same test set. The chosen, capable model was fine-tuned, and it outperformed [11] work’s distilled student (9%) and its one-step teacher (19%). It, however, remained below our own teacher (89%) and their replicated ToT b=5 (82%). The work thus shows that distillation of tree search reasoning into smaller models is possible. Ultimately, the transformer of the reasoning also depends jointly on how diverse and creative the teacher was and the student’s prior capabilities.

6.2 Contributions to the Field

This research offers several concrete advances to the field of reasoning distillation for small language models.

- **Genuine Multi-Step ToT Trace Generation:** Prior work [11] simulated ToT-styled reasoning from a single prompt, and the result had a poorer training example count even after running on 1362 puzzles. Our work constructs a multi-step ToT to a one-step ToT pipeline using a cost-effective model (DeepSeek v4 flash) and achieved 89% solved rate with zero arithmetic errors, externally validated. This proves that a flash-tier model can do quality structured work when a well-engineered framework is designed.
- **Backtracking-Aware Training Data with Dead-End Memory:** Having the full tree in hand for every puzzle, our pipeline produced 1487 distinct examples from only 446 unique puzzles. This is around 10 times larger in volume than the closest comparable work, with an even larger puzzle count. The different backtracking formats teach the student to learn to recover from dead ends rather than memorize correct paths, something that was absent in previous distillation datasets.

- **Code-Grounded Verification for Trace Reliability:** Rather than only relying on the successful parsing of the teacher’s thoughts, forcing it to also produce code in the correct format makes the teacher output more arithmetically correct proposals and makes verification possible mid-tree generation. This eliminates arithmetic hallucinations and parsing errors, and the resulting trees bear high quality and raise the standard of verifiable trace for future distillation pipelines
- **Demonstrated Distillation into a Sub-1B Parameter Model:** Finetuned Qwen2.5-0.5B-Instruct achieved 24% success rate on harder unseen puzzles, rank 901-1000, including backtracking. This is more than double the amount set by prior work (9%), running entirely on local hardware. This confirms that ToT-like search behaviour can be transferred to models that can be run on edge devices through our distillation pipeline, as long as the student has sufficient arithmetic priors.

6.3 Recommendations for Future Work

We can recommend the following future direction of our work:

- **Mix unsolvable problems:** The reason for the fine-tuned student model to backtrack on unsolved problems endlessly was due to not learning when to stop. Thus, some unsolvable combination can be given to the teacher, which can produce actual trees on unsolvable problems, to then distill to our model as a signal on when to give up.
- **Generalization Beyond Arithmetic:** Since the benchmark was done on only one problem, the next step is applying our task-agnostic pipeline on other tasks, like Sudoku, where we can use our teacher and dead-end memory to form traces of “try, fail, recover” format, like proposing digit placement, if it leads to a dead end, then prune it and mark it in memory, etc. Sudoku’s deeper search requirement and comparatively higher backtrack frequency will test whether our backtrack strategy can be effective beyond the three-step arithmetic
- **Multi-Benchmark Dataset Scaling:** Instead of a single problem, fine-tuning on several reasoning task traces will push the student towards more transferable reasoning patterns than just task-specific ones, which would address our work’s current limitation.
- **Higher-Capacity Student Models:** It came from our observation of the dramatic increase in the success rate from capable student models with stronger priors, so further, larger and better arithmetic prior models can be tested with the same rich dataset and compare success rates.

Bibliography

- [1] G. Hinton, O. Vinyals, and J. Dean, *Distilling the knowledge in a neural network*, 2015. arXiv: 1503.02531 [stat.ML]. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, *Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter*, 2019. arXiv: 1910.01108 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [3] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 22 199–22 213.
- [4] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [5] N. Brown, S. Lee, and R. Patel, *Efficient transformer knowledge distillation: A performance review*, Nov. 2023. arXiv: 2311.13657 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2311.13657>
- [6] L. Liu et al., *Think-in-memory: Recalling and post-thinking enable llms with long-term memory*, 2023. arXiv: 2311.08719 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2311.08719>
- [7] J. Long, *Large language model guided tree-of-thought*, 2023. arXiv: 2305.08291 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2305.08291>
- [8] Y. Wang et al., “Self-instruct: Aligning language models with self-generated instructions,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 13 484–13 508. [Online]. Available: <https://aclanthology.org/2023.acl-long.754>
- [9] S. Yao et al., *Tree of thoughts: Deliberate problem solving with large language models*, 2023. arXiv: 2305.10601 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2305.10601>
- [10] D. Zhou et al., *Least-to-most prompting enables complex reasoning in large language models*, 2023. arXiv: 2205.10625 [cs.AI]. [Online]. Available: <https://arxiv.org/abs/2205.10625>
- [11] A. Renduchintala, A. Mahesh, Z. Zhang, Z. Si, S. Meng, and S. Fang, “Babysitting a small language model through one-step tree-of-thoughts knowledge distillation,” 2024, Accessed: 2026-02-05. [Online]. Available: https://www.zichenz.me/project/slm_tot/

- [12] X. Wang et al., *Executable code actions elicit better llm agents*, 2024. arXiv: 2402.01030 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2402.01030>
- [13] D. Jung, W. Zhou, and M. Chen, *Code execution as grounded supervision for llm reasoning*, 2025. arXiv: 2506.10343 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2506.10343>
- [14] M. Kang, J. Jeong, S. Lee, J. Cho, and S. J. Hwang, *Distilling llm agent into small models with retrieval and code tools*, 2025. arXiv: 2505.17612 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2505.17612>
- [15] D. Kim, J. Doo, and M. Seo, *Tslm: Tree-structured language modeling for divergent thinking*, 2026. arXiv: 2601.22688 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2601.22688>
- [16] F. Zaman, *Distilling tree-of-thought reasoning into small language models via prompt-guided imitation*, https://github.com/TheLubricator/one_step_tot_thesis_public, Commit date: 9 June 2026, Jun. 2026.